

Data Science @ scale 1/3

INNER LOOP AND COMPANY CONSTRAINTS

ALEXANDRE GARCIA – DAUPHINE EXECUTIVE MASTER

Motivation

- ▶ Data scientist **expectations and interactions** with other roles
- ▶ Data Scientist **workflow**: on its personal laptop vs in a company
- ▶ Model lifetime: **delivery, production**
- ▶ Maintaining value: **production monitoring and adjustments**

- ▶ Get a big picture to choose career path

Scale matters

Executive Master

- ▶ Few short lived toy projects
- ▶ Dataset is given
- ▶ Dataset is static
- ▶ 1 person
- ▶ No legacy
- ▶ No integration
- ▶ Few constraints

Big Company

- ▶ Many long lived real projects
- ▶ Dataset has to be created, cleansed
- ▶ Data can/will change
- ▶ N teams on different Time zones
- ▶ Legacy by others/lost
- ▶ Integrated by others, multiple times
- ▶ Many constraints

Individual vs System

Before jumping in:

« You do not rise to the level of your goals.
You fall to the level of your systems »

Courses overview

Inner loop

- Constraints, Roles
- Workflow
- Dev environment
- Tooling
- Experiments
- Optimization
- Test
- Communication

Outer loop: delivery

- Infrastructure
- Deployment
- Containers
- Cloud
- GPU
- AI: RAG. Agents

Outer loop: monitoring

- What can happen
- Explainable AI
- Drifts
- Calibration
- MAB, A/B

I – ML/AI in Big companies

- ▶ The bigger the company:
 - ▶ Various business lines: different budgets, capacities, traditions, maturities, tools
 - ▶ Data citizen + specialized roles: what is your role as Data Scientist and what is it not
 - ▶ AI/LLM and LowCode/NoCode challenge current setups with high pressure
- ▶ Constraints for all:
 - ▶ Costs
 - ▶ Security/Compliance
 - ▶ Sourcing

Local vs Global

Local

- ▶ Business knowledge and budgets
- ▶ Geographies
- ▶ Fast track, tactical tradition
- ▶ Harder to maintain
- ▶ Not designed for reuse

Global

- ▶ Critical size to invest in specialized team
- ▶ Platforms
- ▶ Choices of technology
- ▶ Awareness with peers

A solution is a DIY company platform

Security and platformization

- ▶ Single Sign On: France Connect, MyCompany Connect ...
- ▶ Consistent UX (User Experience) internally and externally
- ▶ APIs behind the scenes
- ▶ Offshoring of staff/infrastructure: scalability, cost
- ▶ Security first and everywhere: data, dev, ops -> SecDevOps
- ▶ Regulations, especially in Europe

Code vs LowCode/NoCode

Code

- ▶ For dev
- ▶ For ops
- ▶ For data
- ▶ For sec
- ▶ Advice: master git and github
- ▶ You have full ownership but more maintenance and skills

LowCode/NoCode

- ▶ Most of code and versioning is hidden
- ▶ You can go fast for standard usage, harder to customize and commercial dependency

Some solutions mix code and no/code: Dataiku

3 classical dimensions

- ▶ Business: insights, feature engineering
- ▶ IT: experimenting and shipping
- ▶ Statistical Learning, ... <- Executive Master: know what you use, not just fit/predict
- ▶ Every Data role can be projected on this space

A few roles

- ▶ (BA, Dev, Ops) = traditional software development team
- ▶ Data Analyst, Data Engineer
- ▶ Officers (Compliance, Security, Cloud, Data), Architect
- ▶ Data product manager, DataLab manager
- ▶ **Data Scientist**
- ▶ **MLOps**
- ▶ AI Engineer: Prompt Engineer, RAG developer

Criteria for success

- ▶ DataScience metrics are not overall success:
- ▶ Time to Market
- ▶ Communication
- ▶ Usage Monitoring
- ▶ Data quality
- ▶ Responsiveness to feedback
- ▶ Model monitoring
- ▶ Jobs of many...
- ▶ With conflicting interests sometimes

DevOps

- ▶ Business requests for applications and features
- ▶ Dev codes and tests, CI/CD: continuous integration/deployment
- ▶ Ops runs and monitors the deployed application
- ▶ Dev and Ops are focused on downtime and code quality (bug#)
- ▶ Dev and Ops should be close to optimize Time To Market
- ▶ Dev and Ops share the same tools around GitHub
- ▶ DevOps is more and more **SecDevOps**

Big data, analysts/engineers

- ▶ Business wants to leverage data (reports, ...) and report to regulators. Data can all around the company -> data warehouses
- ▶ Focus is on data freshness, quality. This can be technically challenging for event processing.
- ▶ Data engineers manage the datalakes/mesh/deltalakes (Hadoop, Spark, Databricks), deal with real time events (Apache Kafka). They code and orchestrate pipelines (Airflow, Azure Synapse), aggregate and structure data for use cases. Data scientist and MLOps may have to use similar tools to build the features of their model.
- ▶ Data analysts build reports (PowerBI, Tableau), they can use Spark and SQL. They may create views like a Data scientist may do. They share also some business and data knowledge

From ML perspective: Feature Store (Offline, Online)

Officers

- ▶ The ones that set the rules, all projects should obey the rules
- ▶ Data Officer: ensures data quality, metadata, maps lineage (Collibra). Data and Datasets (group of data) are declared in a catalog. Projects declare usage of datasets, stakeholders are given RO or RW rights on data.
- ▶ Security Officer: ensures all operations are run without vulnerabilities (OS, code, transfers, cloud). Validates vendor solutions feature by feature (long...)
- ▶ Architects: give guidance for standard architectures
- ▶ UX designers: ML frontends must feel the same and be designed with end users

Data Lab Manager

- ▶ Receives requests from many businesses: catalog to store use cases, with their status and metrics
- ▶ Enforces all rules set by officers, many meetings
- ▶ Prioritize use cases depending on feasibility, expected value, risk of not doing
- ▶ Source talented data scientists and ML engineers, offshore ...
- ▶ Control infrastructure costs
- ▶ Represent the company outside, keep aware through peers

Data Scientist – ML Ops

Data Scientist

- ▶ Holds a portfolio of projects (live or dead)
- ▶ Has to speak with business, IT, MLOps
- ▶ Often: works alone with its own habits, feels like he's not a dev or ops but codes better
- ▶ Has to document models and metrics for auditors/regulators
- ▶ Interviews, trainees, student events, data science events

ML Ops

- ▶ Must know ci/cd, containers, cloud
- ▶ Makes sure models can be integrated
- ▶ Very different task constraints (training, inference, tools), adapt resources and minimize cost
- ▶ Production support, monitoring
- ▶ Culture is different from datascience but both should work closely under the same management

AI roles

- ▶ Everyone wants to use it, bottleneck to enforce guardrails and cap cost
- ▶ Entry level is much lower but changes very fast
 - ▶ Prompt engineer, but probably end users some day
 - ▶ RAG devops, workflows (langchain, langgraph, crewAI)
- ▶ But there is still need for data science methods (embedding, chaining of models, model evaluation, quantization) and new requirements for ops (vector databases)
- ▶ In practice, internal access to external LLMs is limited by fear of leakage.
- ▶ Inference with local LLMs is still challenging, SLMs may be the solution

Data project ecosystem

- ▶ Catalogs: datasets, apis, usecases, models
- ▶ External specialized services: OCR, LLM, Voice recording, Speech to Text, ...
- ▶ Internal services at each stage of a usecase:
 - ▶ Tagging interfaces/api for end user
 - ▶ Feedback api on deployed models, integrated in end user frontends
 - ▶ Monitoring dashboard to measure usage
- ▶ Full solutions of business has no dev/ops (for example small business lines like legal teams) or use case is generic.

II Data scientist workflow/tooling

- ▶ Globally: Machine, dev environment
- ▶ Per project:
 - ▶ Environment
 - ▶ Code
 - ▶ Data
 - ▶ Test
- ▶ Experiment loop
- ▶ Optimization, Calibration
- ▶ Maintenance: monitoring, drift

Globally

- ▶ Ops standard is Linux whereas company laptops are generally Windows
- ▶ Do I need a GPU ?
- ▶ How much RAM, disk capacity ? Knowing that i have standard apps running (Outlook, Teams, Web browsers, antivirus ...)
- ▶ Inspired by DevOps, the solution is more and more to be closer to Ops requirements, working remotely on adapted configurations (see that in ML Ops part).
- ▶ Python distribution: Standard or Conda ?
- ▶ Git and GitHub knowledge: branches, pull requests
- ▶ Development best practices, patterns: testing

Different projects

- ▶ 1 major Python version every October, security patches
 - ▶ Many projects with different python versions, how can i do ?
 - ▶ Pyenv or conda enable to have various python installed
 - ▶ Same python but different package versions (pandas, sklearn, ...)
 - ▶ Dependency hell -> Virtual environments
 - ▶ Versions are Fixed in production branches but relaxed in development branch
 - ▶ Poetry helps with dependencies but is very slow on Windows
 - ▶ C++ compilation on Windows requires Build

Python IDE

- ▶ Spyder
 - ▶ will be decommissioned, but pure data scientists still like it
- ▶ Jupyter Notebook/Lab:
 - ▶ not a real IDE
 - ▶ but kernel state persistence is very handy
- ▶ Pycharm (JetBrains)
 - ▶ The reference IDE but VSCode is gaining the edge
- ▶ Visual Studio Code (VSCode, Microsoft)
 - ▶ Backed by Microsoft with an enormous community. Extensions for everything, updated every month. Very strong remoting capabilities
- ▶ Coder: remote coding environments on demand, very trendy

Project code

- ▶ Stored in Git with all configuration except secrets
 - ▶ Interactions through a GitHub enterprise (issues, pull requests)
 - ▶ The bigger the team, higher the required Git skills, code reviews
 - ▶ Conventional commits: enforce good practice and automate release versioning
 - ▶ Automations are more and more started from GitHub (GitHub actions):
 - ▶ Test, release, update documentation, publish metrics (ex DVC)
 - ▶ Beware not to commit any secrets ! Security scans on git repos

Project features - versioning

- ▶ Often neglected but data needs versioning too (align with code, experiment results, time travel)
 - ▶ Folders per version
 - ▶ GitLFS: integration in Git, not always possible
 - ▶ LakeFS: more from bigdata world
 - ▶ DVC: the reference
 - ▶ Xethub: proprietary but has interesting features (notebook state persistence)
 - ▶ Generally, all in one solutions have their own way to deal with versioning and optionally deduplication

Pipeline

- ▶ Have a Standard Pipeline: enforce the same routine on all projects with a one liner: different solutions (DVC, Kedro)
 - ▶ Be sure that data is aligned with code: DVC (sort of git for data)
 - ▶ Experiments: Local results with DVC + store results in a store like MLFlow
 - ▶ Optimize with Optuna/HyperOpt/TensorBoard for AutoML
 - ▶ Bayesian optimization for arbitration between Exploitation and Exploration
 - ▶ Check probabilities calibration, conformal predictions

Tests while developing

- ▶ Parameter space type checked with pydantic
- ▶ Assert presence of keys in dict
- ▶ Asserts shape on dataframe, series, array sizes
- ▶ More explicit tensor sizes with einops
- ▶ Output intermediate results, plots in a folder

Automated tests

- ▶ Changes on dependencies
- ▶ Errors on types (type can have changed, precision...)
- ▶ Mocks to simulate edge cases, errors...
- ▶ Code chaining
- ▶ Generate random data
- ▶ Test on reduced sized, toy model

Experiments

- ▶ Very important to have an automated track/replay of your experiments
 - ▶ DVC has a VSCode extension, same for Tensorboard or Optuna.
 - ▶ Metrics can be pushed to MLFlow which is the reference for ML models catalogs (Azure ML is compliant with ML Flow api)
 - ▶ Kedro has a nice timeline of metrics

Deployment

- ▶ Talk with ML OPs for massive training, inference
- ▶ Inference requires generally an API to wrap models and fulfill company rules (identity) but you can have more integration (Azure ML) to benefit from frontends, ops
- ▶ For specific business requirements, we can deploy an API and even a front end for user acceptance
 - ▶ FastAPI is the reference for Python APIs
 - ▶ Streamlit is widely used for prototyping without HTML or Javascript skills (competitors: Plotly Dash, Bokeh, Nicegui). Voila can webify a Jupyter notebook.

Maintenance (frontier with MLOps)

- ▶ Have automated user feedback, usage monitoring (Elastic Search, Kibana, Graphana, enterprise frontend)
- ▶ Monitor data quality problems
- ▶ Actively retrain -> See session 3

Conclusion for Data science flow

- ▶ Many tools: experiment tracking is absolutely necessary (not only for audit)
 - ▶ Can be quite technical for people with math/stat background, consequently, benefits from DevOps profiles in the team
 - ▶ The technical skills are nonetheless specific to Data Science (numpy, pandas, sklearn, torch, numba ...) and testing is specific
 - ▶ Production/maintenance is part of the workflow
 - ▶ Important to have a reproducible workflow the whole team is comfortable with